

- ▶ Full access to the Qt libraries: PySide bindings allow access to all modules of Qt with library functions running at native (compiled) speed.
- ▶ Shorter code: Python applications – using the same libraries and Qt classes as C++ – have fewer lines of code.
- ▶ Development on the go: As Python is an interpreted language, the runtime already contains all the necessary tools to develop applications, so you don't have to install a compiler, development libraries and header files just to create apps.

Set up the environment

Install the MeeGo notebook either directly on your netbook or in a virtual machine as described earlier. You can either build PySide from source or you can add the following repository to your MeeGo repository list and download pyside from it:

http://repo.pub.meeGo.com/home:/renatofilbo/meeGo_1.1_core/

Sample Code

For our first hello world program, we want to generate a greeting in Python and show it in a QML UI. We do this by :-

- `import sys`

The “sys” module is a Python standard module to access the command line arguments (we need to pass them to the constructor of QApplication).

- `from PySide.QtCore import *`
- `from PySide.QtGui import *`
- `from PySide.QtDeclarative import *`

QtCore contains core objects, like QObject.

QtGui contains QApplication, which we need for the Qt main loop.

QtDeclarativeView provides the QML view.

- `class Hello(QObject):`
- `def get _greeting(self):`
- `return u'Hello, Digit!'`
- `greeting = Property(unicode, get _greeting)`